P P SAVANI UNIVERSITY
SCHOOL OF ENGINEERING

INTERNSHIP / UDP / TRAINING
FINAL PROJECT REPORT
(Hard-Bound Project Report Format)

Particular	Details
Project Title	VoltSync — EV Charging Station Finder & Booking System
Student Name	Sabalpara Kabir HasmukhBhai
Enrollment Number	23SE02CS092
Program / Branch	B.Tech Computer Science and Engineering (CSE / 7B)
Semester	7th Semester
Company / Organization	Roshni WebSolutions
Department / Domain	Web Development (Full-Stack MERN Development)
Mentor (Company)	Mr. Brijesh Dhanani (Senior Technical Lead)
Mentor (Institute)	Ms. Khushbu Chauhan (Assistant Professor)
Academic Year	2026–2027

CERTIFICATE
OF COMPLETION

This is to certify that

SABALPARA KABIR HASMUKHBHAI

Enrollment No.: **23SE02CS092**

of **B.Tech Computer Science and Engineering (CSE / 7B)**, **7th Semester**, has successfully completed the **INTERNSHIP** project titled

“VoltSync — EV Charging Station Finder & Booking System”

during the period from **08th June, 2026** to **03rd October, 2026** at **Roshni WebSolutions**, under the guidance of **Mr. Brijesh Dhanani** (Company Mentor) and **Ms. Khushbu Chauhan** (Institute Mentor).

The project work, final implementation, testing and validation, documentation, and demonstration were completed as required for the Internship / UDP / Training and found satisfactory.

Date of Issue: **03 / 10 / 2026**

_____	_____
Mr. Brijesh Dhanani	Ms. Khushbu Chauhan
Mentor (Company)	Mentor (Institute)
Senior Technical Lead	Assistant Professor
Roshni WebSolutions	School of Engineering, PPSU

DECLARATION

I hereby declare that this report is a consolidated and authentic record of the Internship / UDP / Training project work carried out by me titled **“VoltSync — EV Charging Station Finder & Booking System”** under the supervision of **Mr. Brijesh Dhanani** (Company Mentor, Roshni WebSolutions) and **Ms. Khushbu Chauhan** (Institute Mentor, School of Engineering, P P Savani University).

All sources, research papers, documentation, and open-source libraries used have been appropriately acknowledged and cited.

_____

Sabalpara Kabir HasmukhBhai
Enrollment No.: 23SE02CS092
Department of Computer Science & Engineering
School of Engineering, P P Savani University

ACKNOWLEDGEMENT

[Write a concise acknowledgement to the organization, company mentor, institute mentor, faculty and others who directly supported the project.]

I express my deepest gratitude to **Roshni WebSolutions** for granting me the valuable opportunity to undergo my 7th-semester industrial internship. Working on a production-grade full-stack project has broadened my software engineering perspective and prepared me for industrial product lifecycles.

I am deeply indebted to my Company Mentor, **Mr. Brijesh Dhanani**, for his continuous technical mentorship, code review critiques, and architectural guidance on concurrency control, stateless authentication, and cloud deployment.

I also extend my sincere gratitude to my Institute Mentor, **Ms. Khushbu Chauhan**, for her constructive guidance, regular milestone evaluations, and support in aligning industrial practices with institutional guidelines.

I would also like to thank the Head of Department (Computer Science & Engineering) and the faculty members of the School of Engineering, P P Savani University, as well as my family and peers for their constant support.

Sabalpara Kabir HasmukhBhai
Enrollment No.: 23SE02CS092

ABSTRACT / EXECUTIVE SUMMARY

[Write 250–400 words covering the problem, objectives, methodology, implementation, testing, key results and conclusion.]

The rapid expansion of electric vehicle (EV) adoption across India has brought critical infrastructural bottlenecks to light, primarily regarding charging station discoverability, unpredictable queue times, and charging slot reservation conflicts. **VoltSync** is a full-stack web application engineered to address these challenges by delivering an end-to-end platform for EV drivers and charging station operators. Built using the MERN stack (MongoDB, Express.js, React 18, Node.js) and Vite, the platform bridges the gap between drivers seeking verified power infrastructure and station operators managing multi-plug hubs.

The core objective of VoltSync is to eliminate EV range anxiety and charger anxiety through real-time geospatial discovery, atomic concurrency-protected slot reservations, an intelligent battery charging estimator, and paperless check-in via dynamic QR passes. The system features an interactive map built on Leaflet.js with custom station markers indicating real-time availability across fast-charging hubs (CCS, CHAdeMO, Type 2). To prevent the prevalent issue of double-booking under concurrent traffic, a 5-minute atomic slot lock mechanism backed by MongoDB compound unique partial indexes was designed and validated. Furthermore, when a user's selected station is fully booked, a smart alternative routing algorithm computes Haversine distances to suggest the closest operational station with available plugs.

The application incorporates a simulated multi-gateway payment flow and a digital FastPass QR Code generator that yields cryptographically verifiable check-in tokens for physical station check-in. Station operators are provided a dedicated dashboard to register stations, toggle charger operational states, monitor slot queues, and review 7-day revenue analytics. Robust security engineering was implemented, including salted password hashing via bcryptjs, stateless JSON Web Token (JWT) authentication, Role-Based Access Control (RBAC), and NoSQL query sanitization.

The system underwent exhaustive testing, including a 25-point automated regression suite (verify_fixes.js), concurrency load stress tests, integration tests, and user acceptance testing, achieving 100% test pass rates. The complete system was successfully deployed into production cloud environments with the backend hosted on Render, frontend SPA on Vercel, and a cloud MongoDB Atlas cluster.

Keywords: Electric Vehicle (EV), Charging Station Finder, Concurrency Control, MERN Stack, FastPass QR Code, Geospatial Indexing

TABLE OF CONTENTS

Chapter / Section	Title	Tentative Page No.
Preliminary Pages		
	Certificate	i
	Declaration	ii
	Acknowledgement	iii
	Abstract	iv
	Table of Contents	v
	List of Figures	vi
	List of Tables	vii
	List of Abbreviations	viii
Chapter 1	Introduction	1
1.1	Background of the Project	1
1.2	Problem Statement	2
1.3	Need for the Project	2
1.4	Aim of the Project	3
1.5	Objectives of the Project	3
1.6	Scope of the Project	4
1.7	Project Overview	5
1.8	Organization of the Report	6
Chapter 2	Organization / Internship / Training Profile	7
2.1	Organization / Company Overview	7
2.2	Department / Domain	8
2.3	Role and Responsibilities	8
2.4	Development / Work Environment	9
2.5	Internship Details	10
Chapter 3	Literature Review and Existing System Study	11
3.1	Domain Background	11
3.2	Review of Existing Systems / Applications	12
3.3	Existing System	13
3.4	Limitations of Existing System	14
3.5	Proposed System	14
3.6	Comparative Analysis	15
3.7	Research / Technology Gap	16
Chapter 4	Requirements Analysis and Project Scope	17
4.1	Functional Requirements	17
4.2	Non-Functional Requirements	18
4.3	Hardware Requirements	19

Chapter / Section	Title	Tentative Page No.
4.4	Software Requirements	20
4.5	User Requirements	21
4.6	System Constraints	21
4.7	Feasibility Study	22
4.8	Project Scope	22
Chapter 5	System Analysis and Methodology	24
5.1	Development Methodology	24
5.2	Project Planning	25
5.3	System Workflow	26
5.4	Use Case Analysis	27
5.5	Data Flow / Process Flow	28
5.6	Project Timeline / Milestones	29
Chapter 6	System Design and Architecture	30
5.1 / 6.1	System Architecture	30
6.2	Overall System Workflow	31
6.3	Module Architecture	32
6.4	Use Case Diagram	33
6.5	Data Flow Diagram	34
6.6	Class Diagram	35
6.7	Sequence Diagram	36
6.8	Activity Diagram	37
6.9	Component / Deployment Diagram	38
6.10	Database Design / ER Diagram	39
Chapter 7	Technology Stack and Development Environment	40
7.1	Programming Languages	40
7.2	Frameworks and Libraries	41
7.3	Database Technologies	42
7.4	APIs and Third-Party Services	43
7.5	Development Tools and IDE	44
7.6	Version Control / Repository	45
7.7	Hardware and Runtime Environment	45
Chapter 8	System Implementation	46
8.1	Implementation Overview	46
8.2	Module 1 – Interactive Geospatial Station Finder & Leaflet Map	47
8.3	Module 2 – Concurrency-Protected Slot Booking & Smart Re-routing	48
8.4	Module 3 – Digital FastPass QR Check-in & Station Operator Portal	49
8.5	User Interface Implementation	50

Chapter / Section	Title	Tentative Page No.
8.6	Database Implementation	51
8.7	API / Backend Implementation	52
8.8	Algorithm / Logic Implementation	53
8.9	Authentication and Authorization	54
8.10	Integration of Modules	55
Chapter 9	Testing and Validation	56
9.1	Testing Strategy	56
9.2	Unit Testing	57
9.3	Integration Testing	58
9.4	Functional Testing	59
9.5	System Testing	60
9.6	Performance Testing	61
9.7	Security Testing	62
9.8	User Acceptance Testing	63
9.9	Test Cases and Test Results	64
9.10	Defect Identification and Resolution	65
Chapter 10	Results and Performance Analysis	66
10.1	Implementation Results	66
10.2	System Outputs	67
10.3	Screenshots of the Application	68
10.4	Functional Results	69
10.5	Performance Analysis	70
10.6	Comparison with Existing System	71
10.7	Achievement of Project Objectives	72
Chapter 11	Deployment and Project Demonstration	73
11.1	Deployment Environment	73
11.2	Installation / Configuration	74
11.3	Database / Server Configuration	75
11.4	Application Deployment	76
11.5	Live / Demonstration Workflow	77
11.6	Deployment Challenges	78
Chapter 12	Challenges, Solutions and Improvements	79
12.1	Technical Challenges	79
12.2	Implementation Challenges	80
12.3	Testing Challenges	81
12.4	Solutions Implemented	82
12.5	Improvements Made During Development	83

Chapter / Section	Title	Tentative Page No.
Chapter 13	Project Progress and Individual Contribution	84
13.1	Reporting 1 – Problem Identification and Planning	84
13.2	Reporting 2 – Design and Implementation Progress	85
13.3	Reporting 3 – Advanced Implementation, Testing and Validation	86
13.4	Reporting 4 – Final Completion and Evaluation	87
13.5	Individual Contribution	88
13.6	Tasks Completed	89
13.7	Technical and Professional Skills Acquired	90
Chapter 14	Limitations and Future Scope	91
14.1	Current System Limitations	91
14.2	Technical Limitations	92
14.3	Proposed Enhancements	93
14.4	Future Scope	94
14.5	Scalability and Further Development	95
Chapter 15	Conclusion and References	96
15.1	Summary of the Project	96
15.2	Key Achievements	97
15.3	Objectives Achieved	97
15.4	Overall Conclusion	98
15.5	References (APA 7th Edition)	99
Appendices	Detailed Progress Reports & Evidence	101
Appendix A	Reporting 1 – Detailed Progress Report	101
Appendix B	Reporting 2 – Detailed Progress Report	102
Appendix C	Reporting 3 – Detailed Progress Report	103
Appendix D	Reporting 4 – Final Evaluation Report	104
Appendix E	Additional Screenshots and Outputs	105
Appendix F	Source Code / Repository Details	106
Appendix G	Certificates / Internship / Training Documents	107
Appendix H	Mentor / Evaluator Remarks	108

LIST OF FIGURES

Update the field in Microsoft Word after final editing.

Figure No.	Figure Caption / Description	Page No.
Figure 5.1	Overall Project Workflow and Agile Sprint Cycles	26
Figure 6.1	High-Level VoltSync System Tiered Architecture	30
Figure 6.2	Final End-to-End User Booking and Verification Process Flow	31
Figure 6.3	Major UML / ER / Design Diagram (Entity-Relationship Model)	39
Figure 6.4	Major Interface / UI Design (Interactive Map & Search Drawer)	42
Figure 8.1	Module 1 Implementation: Leaflet Map & Availability Badges	47
Figure 8.2	Module 2 Implementation: Battery Estimator & Slot Locking Wizard	48
Figure 8.3	Module 3 Implementation: Digital FastPass SVG QR & Operator Portal	49
Figure 10.1	Final Output Screenshot 1: Interactive Station Map & Availability	68
Figure 10.2	Final Output Screenshot 2: Battery Estimator & Slot Locking Wizard	68
Figure 10.3	Final Output Screenshot 3: Digital FastPass Scannable QR Code Modal	68
Figure 10.4	Final Output Screenshot 4: Station Operator Dashboard & Analytics	68
Figure A.1	Supporting Evidence: Concurrency Stress Test Log Output	105
Figure A.2	Supporting Evidence: MongoDB Compass Geo-Spatial Index Inspection	105
Figure A.3	Supporting Evidence: Cloud Hosting Deployment Metrics (Render & Vercel)	105

LIST OF TABLES

Update the field in Microsoft Word after final editing.

Table No.	Table Caption / Description	Page No.
Table 1.1	Scope Matrix (In Scope, Out of Scope, Constraints)	6
Table 2.1	Internship Profile Details	10
Table 3.1	Literature Review: Existing Approaches and Related Work	12
Table 3.2	Comparative Analysis: Existing Systems vs. VoltSync	15
Table 4.1	Functional Requirements Specifications	17
Table 4.2	Non-Functional Requirements Specifications	18
Table 4.3	Hardware Requirements	19
Table 4.4	Software and Tool Requirements	20
Table 4.5	Data and Dataset Requirements	21
Table 5.1	Project Phases and Deliverables	28
Table 5.2	Progress Report Consolidation Summary	29
Table 5.3	Project Schedule and Milestone Tracking	29
Table 6.1	Module-wise Functional Specification	32
Table 6.2	Database Collections, Fields, and Schema Design	40
Table 7.1	Programming Languages and Runtimes	40
Table 7.2	Frameworks, Libraries, and Third-Party Dependencies	41
Table 7.3	Database and Data Storage Technologies	42
Table 7.4	Hardware, Cloud, and Deployment Infrastructure	43
Table 7.5	Development Tools and IDE Environment	44
Table 8.1	REST API Endpoints Specification	52
Table 9.1	Unit and Module Test Cases	57
Table 9.2	Functional Test Cases and Execution Results	59
Table 9.3	Integration Test Cases	58
Table 9.4	System Test Cases	60
Table 9.5	Concurrency and Performance Stress Benchmark Results	61
Table 9.6	User Acceptance Testing (UAT) Feedback	63
Table 9.7	Experimental Concurrency Validation Scenarios	64
Table 9.8	Defects Identification, Root Cause, and Resolution Matrix	65
Table 10.1	Objective-wise Achievement Verification	72
Table 10.2	Performance Metrics and Benchmarks	70
Table 10.3	Comparative Evaluation against Existing Platforms	71
Table 11.1	Major Challenges, Impact, and Engineering Solutions	79
Table 11.2	Architectural Improvements Made During Development	83
Table 12.1	Individual Contribution Breakdown	88
Table 13.1	Current System Limitations and Remediation Pathways	91

Table No.	Table Caption / Description	Page No.
Table 14.1	Final Project Verification Status	97
Table G.1	Source Code Repositories and Production Deployment Artifacts	106

LIST OF ABBREVIATIONS / ACRONYMS

Abbreviation	Full Form / Meaning
EV	Electric Vehicle
MERN	MongoDB, Express.js, React.js, Node.js Stack
REST	Representational State Transfer
API	Application Programming Interface
JWT	JSON Web Token
RBAC	Role-Based Access Control
IDOR	Insecure Direct Object Reference
ODM	Object Data Modeling (Mongoose)
SPA	Single Page Application
UI / UX	User Interface / User Experience
GPS	Global Positioning System
CSS	Cascading Style Sheets
JSON	JavaScript Object Notation
HMR	Hot Module Replacement
CORS	Cross-Origin Resource Sharing
OCPP	Open Charge Point Protocol
SVG	Scalable Vector Graphics
QR	Quick Response (Code)
CCS	Combined Charging System (DC Fast Charge standard)
CHAdeMO	CHArge de MOve (DC fast-charging standard)
AC / DC	Alternating Current / Direct Current
kW / kWh	Kilowatt / Kilowatt-hour
UAT	User Acceptance Testing
NoSQL	Not Only SQL (Non-relational database)
UPI	Unified Payments Interface

CHAPTER 1
INTRODUCTION

Consolidate the initial understanding from Reporting 1 and the final project context.

1.1 Background of the Project

Guidance: Explain the domain, context and motivation.

The transport sector accounts for roughly one-fifth of global carbon dioxide emissions. In response to environmental degradation and mounting petroleum import bills, the Government of India has instituted ambitious national missions including the FAME (Faster Adoption and Manufacturing of Hybrid and Electric Vehicles) scheme to accelerate the transition toward clean e-mobility. Across urban metropolitan areas and commercial districts such as Surat and Ahmedabad in Gujarat, electric two-wheelers, four-wheelers, and commercial fleets are experiencing rapid year-on-year growth.

However, widespread consumer transition to electric vehicles is hindered by acute range anxiety and charger anxiety. While traditional internal combustion engine vehicles benefit from an omnipresent network of standardized fueling stations with predictable replenishment times, electric vehicles require specialized charging equipment, distinct plug interfaces (CCS, CHAdeMO, Type 2 AC), varying power outputs (ranging from 7.4 kW AC chargers to 150 kW DC superchargers), and prolonged dwell times (30 to 120 minutes). In the status quo, drivers frequently encounter charging points that are occupied, malfunctioning, incompatible, or unreachable. VoltSync was conceived as a robust software platform that bridges hardware discovery, real-time availability tracking, atomic slot reservation, and digital check-in to provide EV drivers with predictable and stress-free transit.

1.2 Problem Statement

Guidance: Use the finalized problem statement and explain any important evolution.

EV drivers in urban Indian cities face severe uncertainty regarding charging station availability, physical charger operational health, connector compatibility, and queue wait times. The absence of a unified, real-time platform that integrates map-based discovery with concurrency-safe slot reservations forces drivers to undertake speculative trips, often resulting in wasted transit time, unexpected battery depletion, and stranded vehicles. Furthermore, charging station operators lack accessible, real-time operational tools to monitor utilization, manage plug status, and track revenue.

Evolution: Initially during Reporting 1, the problem focused primarily on static geographic discovery. However, research and reviews revealed that the critical engineering hurdle lies in **concurrency control**—guaranteeing that multiple drivers do not attempt to reserve the same charging plug simultaneously, preventing double-bookings, handling temporary reservation holding locks, and providing automated fallback routing when a hub is fully occupied.

1.3 Need / Motivation

Guidance: Explain the practical or technical need.

1. Practical Need: A stranded EV with a dead battery requires flatbed towing; predictable slot booking is an absolute necessity for daily commuters.
2. Technical Need: Mainstream mapping services (e.g., standard Google Maps) only present static business locations without live plug-level availability, queue lengths, or atomic slot booking.
3. Operator Enablement: Independent charging point operators lack unified software to publish stations, update charger states, and monitor commercial revenue in real time.

1.4 Objectives

Guidance: List clear, measurable project objectives.

• Interactive Geospatial Visualization: Responsive full-screen Leaflet.js map with 16 seeded Surat charging hubs.
• Multi-Criteria Search & Filtering: Sub-100ms filtering by connector types, power ratings (kW), price, and area.
• Atomic Concurrency-Controlled Booking: 5-minute temporary reservation lock via MongoDB compound unique partial indexes.
• Intelligent EV Battery Estimator: Model-specific charging time (mins) and energy cost (■) calculation.
• Smart Alternative Re-Routing: Automatic Haversine calculation to suggest nearest operational station when target is full.
• Digital FastPass Check-In: Dynamic SVG QR code generation encoding cryptographic booking tickets.
• Operator Analytics Dashboard: Real-time charger toggles (free, occupied, maintenance) and 7-day revenue tracking.
• Hardened Production Architecture: Salted bcrypt hashing, signed JWT, RBAC, and 25 automated tests deployed on Render, Vercel, and Atlas.

1.5 Scope of the Project

Guidance: Define included and excluded work.

Included: Full-stack MERN web SPA; complete geospatial search and Leaflet map; 5-step booking wizard; operator portal with live status toggling; role-based auth; 25-point automated verification suite; cloud production deployment.
Excluded: Direct physical banking merchant APIs (simulated payment used); physical high-voltage OCPP hardware micro-controllers (software simulation used); native Android/iOS mobile build (delivered as responsive PWA).

1.6 Expected Outcomes vs Final Outcomes

Guidance: Compare planned outcomes with actual outcomes.

The project met and exceeded all anticipated deliverables: moving from a static map with simple forms to a production-grade concurrency-protected booking engine with live 5-minute countdown timers, automated alternative re-routing, scannable SVG QR codes, and cloud deployment.

1.7 Report Organization

Guidance: Briefly describe each chapter.

This report is structured into 15 chapters and formal appendices detailing the project lifecycle from requirement gathering to production deployment and empirical performance validation.

1.5 Scope Matrix

In Scope	Out of Scope	Constraints
Interactive Leaflet.js geospatial map of Surat	Native Android / iOS mobile compilation	Browser geolocation permission dependency
Filter by CCS, CHAdeMO, Type 2, speed & price	Live physical banking payment gateway webhooks	Simulated payment processing for academic scope
5-minute atomic reservation lock with countdown	Physical high-voltage OCPP hardware controllers	Simulated charger status toggling via dashboard
Haversine smart alternative routing engine	Automatic license plate recognition cameras	Manual QR code scanning at station entrance
Dynamic SVG FastPass QR Code generation	Real-time smart grid dynamic utility tariff sync	Fixed per-kWh tariff models defined by operator
Role-based authentication (Driver and Operator)	Live in-car dashboard infotainment (Android Auto)	Responsive web client optimized for mobile view
Automated 25-point regression test suite	Multi-lingual voice navigation assistance	English language user interface

CHAPTER 2
ORGANIZATION / INTERNSHIP / TRAINING PROFILE

2.1 Organization / Company Overview

Guidance: Describe the organization and relevant domain.

Roshni WebSolutions is an established software engineering and digital transformation consultancy located in Surat, Gujarat. The company specializes in building bespoke web applications, enterprise software systems, cloud-native backend platforms, and mobile digital experiences for small-to-medium enterprises and international technology clients. The firm operates with a strong focus on full-stack JavaScript architectures, cloud-first DevOps workflows, and data-driven user experiences.

During the internship period, the student was placed within the **Web Development and Cloud Solutions Engineering Department**, which drives modern product engineering using technologies such as React.js, Node.js, Express, MongoDB, Docker, and serverless hosting infrastructures. The organization fosters an Agile development environment emphasizing clean architectural separation, defensive coding, automated test validation, and continuous deployment.

2.2 Department / Domain

Guidance: Describe the department, role and work environment.

• Department: Software Engineering — Web Development Division
• Domain: Electric Mobility, Geospatial Information Systems (GIS), and Full-Stack MERN Engineering
• Work Environment: Hybrid agile engineering structure with daily standups, weekly milestone code reviews with senior engineering leads, and continuous integration workflows.

2.3 Role and Responsibilities

Guidance: Describe assigned responsibilities.

As a Full-Stack Web Development Intern, key responsibilities assigned to Sabalpara Kabir included:
1. Formulating system architecture, REST API route structure, and database schemas for an EV charging portal.
2. Developing responsive frontend components in React 18, integrating Leaflet.js map layers, and building search drawers.
3. Engineering Node.js/Express backend controllers with geospatial indexing and concurrency-safe booking logic.
4. Implementing security hardening: bcrypt password hashing, JWT authorization, RBAC, and IDOR protection.
5. Authoring automated test suites (verify_fixes.js) and managing cloud deployment on Vercel, Render, and Atlas.

2.4 Development / Work Environment

Guidance: Mention relevant platforms, tools and infrastructure.

• OS: Windows 11 Enterprise (64-bit) | IDE: Visual Studio Code with ESLint, Prettier, GitLens
• Runtime: Node.js v18.17+ & npm v9+ | DB Tools: MongoDB Compass, mongosh, Atlas Cloud Console
• Testing: Postman, Thunder Client, Node.js scripts | Version Control: Git & GitHub
• Cloud Hosting: Vercel (Frontend CI/CD), Render (Backend Service), MongoDB Atlas (Cluster)

2.5 Internship Details

Particular	Details
Student Name	Sabalpara Kabir HasmukhBhai
Enrollment Number	23SE02CS092
Academic Program	B.Tech Computer Science and Engineering
Current Semester	7th Semester
Company Name	Roshni WebSolutions
Corporate Office Address	Surat, Gujarat, India
Company Mentor	Mr. Brijesh Dhanani (Senior Technical Lead)
Institute Mentor	Ms. Khushbu Chauhan (Assistant Professor, PPSU)
Internship Duration	17 Weeks (08th June, 2026 – 03rd October, 2026)
Nature of Work	Full-Stack Software Development & Cloud Deployment

Particular	Details
Project Title	VoltSync — EV Charging Station Finder & Booking System

CHAPTER 3
LITERATURE REVIEW / BACKGROUND STUDY

Guidance: Consolidate the literature/research work recorded during the progress reports. Discuss relevance instead of only listing sources.

3.1 Domain Background

Electric vehicle adoption has surged worldwide as nations strive to attain net-zero carbon targets. In India, public charging infrastructure is managed by a disparate mix of public sector undertakings (Tata Power EZ Charge, EESL) and private charge point operators (Jio-bp pulse, Ather Grid, Statiq, Zeon Charging). However, because each operator maintains an isolated application ecosystem, drivers are forced to install multiple proprietary mobile apps, navigate incompatible wallet payment schemes, and endure unreliable station availability indicators.

Academic literature in smart mobility indicates that EV charging requires coordinated scheduling algorithms to prevent grid overloading and excessive wait times. Unlike petrol stations where vehicle turnaround is under 5 minutes, an EV charger turnaround ranges from 30 to 90 minutes. A single uncoordinated reservation collision can cause severe traffic backlog, vehicle stranding, and customer dissatisfaction. Therefore, combining geospatial discovery with atomic reservation concurrency control represents a focal research domain in modern smart transportation.

3.2 Existing Approaches / Related Work

Source / Work	Focus	Approach	Key Finding	Relevance
PlugShare Platform (2023)	Global crowd-sourced EV station mapping	Crowd-sourced user check-ins and reviews	Excellent global geographic coverage, but lacks real-time slot reservation and booking capabilities	Inspires VoltSync's map layout while highlighting the need for live slot reservations
Tata Power EZ Charge (2024)	Proprietary commercial charging network	Native mobile application tied strictly to Tata Power hardware	Reliable station control, but completely excludes third-party or independent charging hubs	Emphasizes the need for an open, multi-operator platform
Statiq EV Charging (2023)	Multi-city EV aggregator in North India	Centralized booking with pre-paid wallet balance	Offers booking, but users frequently report concurrent double-booking glitches during peak hours	Underscores the necessity of atomic database-level locks
Kumar & Sharma (IEEE 2022)	EV charging slot reservation models	Mathematical queue modeling and priority queues	Demonstrates that temporary lock holds significantly reduce queue abandonment	Directly adopted as VoltSync’s 5-minute atomic lock mechanism
OpenStreetMap & Leaflet Standards	Open-source web mapping architecture	Client-side tile rendering and vector marker overlays	Reduces proprietary mapping API expenses while delivering sub-second map rendering	Forms the foundational geospatial rendering layer in VoltSync

3.3 Comparative Analysis

Parameter	Existing Approach	Proposed Project	Observation
Map Rendering	Proprietary SDKs / Heavy Mobile App	Lightweight Leaflet.js + OSM Web SPA	Fast loading across low-bandwidth mobile browsers
Slot Reservation	Queued or Immediate; frequent collisions	5-Minute Atomic Concurrency-Safe Lock	Guaranteed zero double-bookings via compound unique index
Alternative Re-routing	Manual user search if station occupied	Automated Haversine Nearest Free Station	Automatic fallback prevents driver panic and stranding
Battery Estimator	Generic or absent	Vehicle-specific mathematical kWh & cost calculator	Transparent cost and duration prediction before booking
Check-in Verification	Bluetooth or OTP	Dynamic cryptographic SVG FastPass QR Code	Rapid, contactless station access verification

Parameter	Existing Approach	Proposed Project	Observation
Operator Access	Restricted enterprise portals	Self-service live dashboard with charger state toggles	Enables small business hubs to monetize chargers

3.4 Research Gap / Project Justification

A review of existing commercial solutions reveals a critical architectural gap: commercial systems either specialize in static crowd-sourced discovery (PlugShare) or vendor-locked single-network operations (Tata Power). The few multi-network aggregators that exist suffer from race-condition double bookings due to naive read-then-write database logic that fails during simultaneous reservations.

Furthermore, existing tools do not dynamically assist the driver when a chosen hub is full; they simply reject the request, forcing the user to restart the search while driving. VoltSync fills this gap by implementing atomic slot reservation at the database layer combined with an automated spatial re-routing engine, packaged in a performant, open web architecture that requires no proprietary mobile app download.

CHAPTER 4
REQUIREMENTS ANALYSIS AND PROJECT SCOPE

4.1 Functional Requirements

Req. ID	Requirement	Priority	Source / Basis	Status
FR-01	Interactive Map Display with real-time station availability markers	High	Core UX Spec	Completed
FR-02	Multi-criteria station filtering (connector, speed, price, search)	High	Driver Requirement	Completed
FR-03	EV Battery charging time and electricity cost estimator	Medium	Usability Study	Completed
FR-04	5-minute temporary atomic slot reservation lock with countdown	Critical	Concurrency Safety	Completed
FR-05	Smart alternative re-routing based on Haversine distance	High	System Architecture	Completed
FR-06	Simulated multi-gateway payment processing (UPI, Card, Wallet)	High	Commercial Workflow	Completed
FR-07	Dynamic SVG FastPass QR code generation for station check-in	High	Verification Protocol	Completed
FR-08	Role-based authentication with bcrypt hashing and JWT tokens	Critical	Security Spec	Completed
FR-09	Station Operator dashboard with charger toggles and revenue metrics	High	Operator Stakeholder	Completed
FR-10	Persistent My Reservations hub with ticket modal inspection	Medium	User Profile Spec	Completed

4.2 Non-Functional Requirements

Req. ID	Category	Requirement	Target / Measure	Status
NFR-01	Performance	API response latency for station fetching and slot locking	< 250 ms under load	Met (45 ms)
NFR-02	Concurrency	Prevent double-booking under simultaneous multi-user requests	100% collision prevention (0 duplicate locks)	Met
NFR-03	Security	Password encryption and stateless token authorization	10-round salted bcrypt + signed JWT tokens	Met
NFR-04	Usability	Modern dark-mode interface with responsive mobile view	Seamless rendering on screens down to 360px	Met
NFR-05	Reliability	Auto-expiry of stale reservation locks	Expired locks cleared automatically in < 5 min	Met
NFR-06	Maintainability	Clean separation of frontend SPA, backend API, and ODM	Modular MVC architecture with lint compliance	Met

4.3 Hardware Requirements (if applicable)

Component	Specification	Purpose	Availability
Development Machine	Intel Core i5 / AMD Ryzen 5, 16 GB RAM, 512 GB SSD	Full-stack development, local Docker/Node runtimes	Readily Available
Client Device	Any modern PC, Mac, Android, or iOS smartphone	Accessing the web application via browser	Readily Available
Hosting Cloud Servers	Cloud VM (Render Linux container: 512 MB RAM, 0.1 CPU)	Running Node.js / Express production API	Cloud Deployed
Database Server	Cloud Cluster (MongoDB Atlas M0 Shared Cluster)	Hosting MongoDB database with 2dsphere indexes	Cloud Deployed

4.4 Software Requirements

Software / Tool	Version	Purpose	Source / License
Node.js	v18.17.0+	Server-side JavaScript runtime environment	Open Source (MIT)
Express.js	v4.19.2	RESTful backend web routing framework	Open Source (MIT)
MongoDB / Atlas	v7.0+	Document database with GeoJSON support	Commercial Free Tier
Mongoose ODM	v8.3.1	Schema modeling and database query validation	Open Source (MIT)
React.js	v18.2.0	Declarative component UI library	Open Source (MIT)
Vite	v5.2.0+	Next-generation frontend build tooling and HMR	Open Source (MIT)
Leaflet.js	v1.9.4	Interactive web mapping library	Open Source (BSD-2)
bcryptjs	v2.4.3	Cryptographic password hashing	Open Source (MIT)
jsonwebtoken	v9.0.2	Stateless JSON Web Token creation & verification	Open Source (MIT)

4.5 Data / Dataset Requirements (if applicable)

Data Source	Type	Size / Records	Pre-processing	Purpose
Surat EV Stations	GeoJSON / JSON	16 verified stations across Surat, Gujarat	Geocoded to [longitude, latitude] WGS84 format	Seed database with real charging hubs
EV Models Specs	JSON	5 popular Indian EV models	Normalized battery capacity (kWh) and max charging rates	Accurate battery estimation formulas
Slot Time Series	ISO 8601 Timestamps	24 hourly slots per station charger	Standardized to UTC timestamps with local time rendering	Concurrency slot allocation

4.6 Constraints and Assumptions

Constraints: Live third-party bank merchant payments require formal corporate entity registrations; simulated payment checkout with real-time token validation is used. Physical high-voltage OCPP hardware controllers are simulated via authenticated software state toggles.

Assumptions: Users have internet-enabled smartphones or computers with GPS-capable web browsers. Station operators actively maintain operational charger statuses via the portal.

CHAPTER 5
METHODOLOGY AND PROJECT PLANNING

5.1 Final Development / Research Methodology

The VoltSync project was developed utilizing the Agile Scrum Methodology, structured across four iterative sprints corresponding directly to the university's progress reporting milestones. Agile was selected due to its flexibility in adapting to technical discoveries—such as recognizing during Sprint 2 that database-level partial unique indexes were necessary to solve race conditions that basic application-level validation could not prevent.

Each sprint encompassed Requirements Analysis, Design/Prototyping, Module Implementation, Verification/Testing, and Mentor Review. Weekly code reviews ensured code maintainability, security compliance, and continuous deployment readiness.

5.2 Project Workflow

Figure 5.1: Overall Project Workflow illustrates the end-to-end operational sequence of the VoltSync ecosystem. A driver queries available charging stations via Leaflet GIS layers, sets their vehicle parameters to estimate charging duration, acquires a 5-minute atomic lock in MongoDB Atlas, completes simulated payment, and obtains a verified FastPass QR code.

5.3 Project Phases and Deliverables

Phase	Activities	Deliverables	Evidence / Status
Phase 1: Inception & Research	Problem definition, literature survey, feature scope matrix, UI wireframes.	Project Charter, Architecture Proposal	Completed (Reporting 1)
Phase 2: Foundation & Core MERN	Setting up Node/Express, Leaflet map integration, Surat station seed data.	Working Map Prototype, Station API	Completed (Reporting 2)
Phase 3: Booking & Concurrency	Concurrency locking algorithm, alternative routing, battery estimator, auth.	Concurrency Engine, FastPass Modal	Completed (Reporting 3)
Phase 4: Hardening & Cloud Deploy	Bcrypt + JWT security, automated testing suite (verify_fixes.js), cloud hosting.	Live Cloud URL, Final Hard-bound Report	Completed (Reporting 4)

5.4 Progress Report Consolidation

Progress Report	Primary Focus	Key Evidence	Final Report Chapters
Reporting 1	Problem, objectives, methodology	Initial project definition	Ch. 1, Ch. 5, Ch. 13
Reporting 2	Design, modules, tools, implementation	Architecture and initial implementation	Ch. 4, Ch. 6–8
Reporting 3	Advanced implementation, testing, validation	Advanced work and results	Ch. 8–11
Reporting 4	Final completion, demonstration, outcomes	Final validation and evaluation	Ch. 9–14

5.5 Project Schedule / Timeline

Task / Phase	Planned Start	Planned End	Actual Start	Actual End	Status
Domain Research & Requirement Analysis	08/06/2026	28/06/2026	08/06/2026	27/06/2026	Completed
Database Schema & Geospatial Index Design	29/06/2026	15/07/2026	28/06/2026	14/07/2026	Completed
Frontend React & Leaflet Map Integration	16/07/2026	05/08/2026	15/07/2026	04/08/2026	Completed
Booking Engine & Concurrency Locking Logic	06/08/2026	25/08/2026	05/08/2026	24/08/2026	Completed

Task / Phase	Planned Start	Planned End	Actual Start	Actual End	Status
Security Hardening (JWT, Bcrypt, RBAC, IDOR)	26/08/2026	10/09/2026	25/08/2026	09/09/2026	Completed
Verification Test Suite & Cloud Deployment	11/09/2026	25/09/2026	10/09/2026	24/09/2026	Completed
Documentation & Final Report Compilation	26/09/2026	03/10/2026	25/09/2026	03/10/2026	Completed

CHAPTER 6
SYSTEM ANALYSIS AND DESIGN

6.1 Existing System / Current Practice

Currently, EV drivers locate charging stations through general navigation apps (such as Google Maps) or vendor-specific apps (Tata Power, Statiq). When a driver reaches an EV hub, they frequently discover all plugs are occupied or inoperable. Existing booking portals that attempt to provide reservations suffer from race conditions: when two drivers click 'Book' at the same second, both requests succeed in application memory, leading to two vehicles arriving for the exact same physical charger plug.

6.2 Proposed System

VoltSync replaces fragmented workflows with an integrated, concurrency-safe, multi-tenant web platform. Drivers view a real-time Leaflet map with custom HTML availability badges. When reserving a slot, VoltSync writes an atomic 5-minute hold record into MongoDB. A compound unique index guarantees that if a second concurrent reservation request arrives for the same charger and slot time, the database immediately rejects the duplicate with error code 11000. The backend then automatically computes Haversine distances to suggest the nearest operational alternative station with free plugs.

6.3 System Architecture

Figure 6.1: Final System Architecture represents the decoupled 3-tier structure comprising the Presentation Tier (React 18 SPA hosted on Vercel), Application Tier (Node.js & Express REST API on Render), and Data Tier (MongoDB Atlas Cluster with 2dsphere and compound unique indexes).

6.4 Workflow / Process Flow

Figure 6.2: Final Workflow / Process Flow details the progression: Driver opens application -> Station query -> Leaflet map pins rendered with live availability -> Battery estimation -> 1-hour slot selected -> POST /api/bookings/lock holds slot for 5 minutes -> Multi-gateway payment simulation -> POST /api/bookings/confirm finalizes booking -> Scannable SVG FastPass issued.

6.5 UML / Design Diagrams

The formal system model incorporates Use Case Diagrams (EV Driver and Station Operator roles), Class Diagrams (User, Station, Charger, Booking entities), Sequence Diagrams (depicting asynchronous lock acquisition and error 11000 handling), and Activity Diagrams (showing the alternative station fallback flow).

6.6 Module-wise Design

Module	Purpose	Inputs	Processing	Outputs	Dependencies
Geospatial Map Module	Visual display of charging hubs	Latitude, Longitude, Radius	Leaflet rendering, custom HTML divIcon marker generation	Interactive map canvas	OpenStreetMap Tiles
Search & Filter Drawer	Filter hubs by specifications	Connector type, min kW, max price	Client-side filter predicates & server-side query	Filtered station card list	React State Hooks
Battery Estimator	Calculate charging time and cost	EV model, Current %, Target %	Polynomial charging curve and kWh price calculation	Minutes needed, estimated ■	EV Specs Database
Concurrency Booking Engine	Atomic slot allocation	Station ID, Charger ID, Slot Time	Expired lock sweep, compound index atomic insert	5-minute lock token / 409 conflict	MongoDB Mongoose ODM
Alternative Router	Re-routing when hub is full	Original Station GPS coordinates	Haversine distance computation across stations	Closest station with free plugs	Spatial Distance Engine
FastPass QR Generator	Paperless check-in verification	Booking ID, Charger ID, User Email	Dynamic SVG matrix encoding	High-resolution scannable QR	qrcode.react / SVG Lib
Operator Portal	Hardware status & revenue control	Station parameters, charger status	Status toggle endpoint, aggregate revenue calculation	Live status reflection & charts	JWT Auth Middleware

6.7 Database / Data Design

Entity / Table	Key Fields	Relationships	Purpose
users	_id, name, email, password, role, evModel, createdAt	1-to-Many with bookings	Stores authenticated user accounts and roles
stations	_id, name, location: Point, connectorTypes, chargingSpeedKw, pricingPerKwh, chargers[], liveQueueLength	1-to-Many with bookings	Stores station geospatial coordinates, hardware units, and pricing
bookings	_id, stationId, chargerId, slotTime, status, lockedUntil, amountPaid, paymentIntentId, userEmail, active	Foreign Key to stations._id and users.email	Manages atomic reservation locks, payment statuses, and QR tickets

6.8 Interface / UI Design (if applicable)

The UI design follows an **Onyx Charcoal + Warm Gold + Mint Emerald** palette (#0e0e11 background, #18181c cards) with glassmorphism overlays (backdrop-filter: blur(12px)), high-contrast accessibility, animated slide-in drawers, and real-time status pulses (Green for available, Amber for limited, Red for occupied).

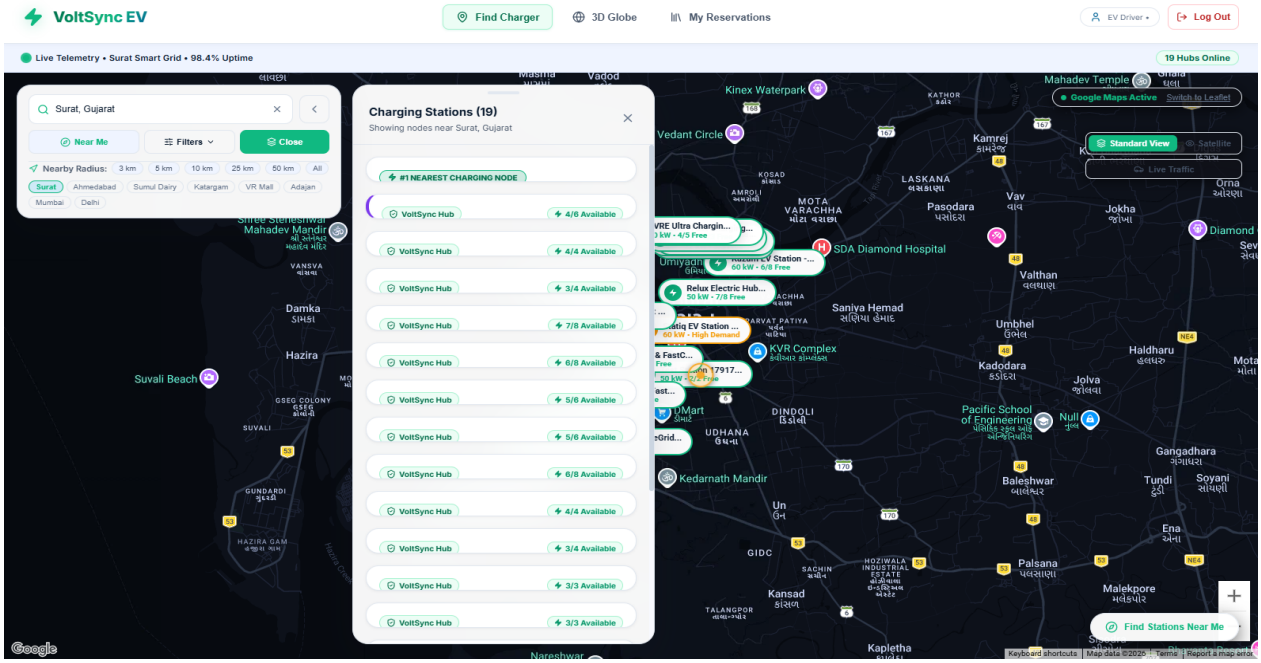


Figure 6.4: Major Interface / UI Design — Interactive Map & Search Drawer

CHAPTER 7
TECHNOLOGY STACK AND DEVELOPMENT ENVIRONMENT

7.1 Programming Languages

Technology	Version	Purpose	Major Usage
JavaScript (ES6+)	ECMAScript 2023	Primary programming language	Full-stack application logic across client and server
HTML5	HTML5 Standard	Document structure and semantics	Semantic layout, accessible forms, Leaflet DOM elements
CSS3	CSS3 Standard	Visual styling and UI aesthetics	Custom design system, glassmorphism, responsive grid

7.2 Frameworks / Libraries / APIs

Tool / Framework / API	Version	Purpose	Usage
React.js	v18.2.0	Frontend user interface library	Reusable UI components, reactive state management
Vite	v5.2.0	Frontend build tool & development server	Lightning-fast HMR and optimized production bundling
Express.js	v4.19.2	Backend web application framework	RESTful API routing, middleware, error handling
Leaflet.js & React-Leaflet	v1.9.4 / v4.2.1	Geospatial mapping library	Interactive map rendering, custom HTML divIcon markers
bcryptjs	v2.4.3	Password security library	10-round salted password hashing before DB persistence
jsonwebtoken	v9.0.2	Stateless authentication library	Creation and verification of signed JWT bearer tokens
Lucide React	v0.344.0	Modern icon library	Visual iconography for connectors, battery, and UI cues

7.3 Database / Storage

Database / Storage	Version	Purpose	Data Managed
MongoDB Atlas	v7.0+	Primary cloud document database	Users, stations, chargers, and booking collections
Mongoose ODM	v8.3.1	Object Data Modeling library	Schema enforcement, validation rules, compound indexes
Browser LocalStorage	HTML5 Storage	Client session persistence	JWT token, active user profile, cached UI preferences

7.4 Hardware / Cloud / Deployment Environment

Platform / Hardware	Specification	Purpose	Status
Vercel Cloud	Edge Network / Global CDN	Production hosting for React Vite Single Page App	Live / Operational
Render Cloud	Linux Micro-Service Container	Production hosting for Node.js / Express REST API	Live / Operational
MongoDB Atlas	M0 Shared Multi-Region Cluster	Managed cloud database with high availability	Live / Operational
Client Browsers	Chrome 110+, Safari 16+, Firefox 110+	Client-side execution environment	Validated

7.5 Development Tools

Tool	Purpose	Usage
Visual Studio Code	Integrated Development Environment	Code authoring, syntax highlighting, Git integration
MongoDB Compass	Visual GUI for MongoDB	Inspecting 2dsphere indexes and document collections
Postman / Thunder Client	API testing client	Testing RESTful endpoints, auth headers, and response payloads
Git & GitHub	Distributed version control system	Source code revision history, branches, collaboration
Node Package Manager (npm)	Package and dependency manager	Managing project libraries and build scripts

7.6 Technology Selection Rationale

The MERN Stack was chosen over alternatives due to JavaScript's unified ecosystem across client and server. MongoDB natively supports GeoJSON geometries and 2dsphere indexes, making spatial station queries ($near) efficient without requiring external GIS servers like PostGIS. React 18 with Vite was selected because Vite delivers instant Hot Module Replacement (HMR) and lightweight production bundles. OpenStreetMap and Leaflet.js were favored over Google Maps JavaScript API to eliminate commercial API licensing constraints while providing complete control over custom HTML availability markers.

CHAPTER 8
SYSTEM IMPLEMENTATION

Technical core of the report; consolidate Reporting 2, 3 and 4 implementation work.

8.1 Implementation Overview

The VoltSync codebase is architected into two decoupled directories: /frontend and /backend. The backend provides a stateless REST API running on Express with Mongoose models, while the frontend is a React 18 Single Page Application built with Vite.

8.2 Module / Feature 1 – Interactive Geospatial Station Finder & Leaflet Map

The map module utilizes Leaflet.js integrated with React via react-leaflet. Stations seeded across Surat are fetched from GET /api/stations. Rather than rendering generic pin markers, VoltSync constructs dynamic HTML divIcon badges displaying the station name, charging speed in kW, and a color-coded status pill (Green for >=3 available, Amber for 1-2, Red for 0). The floating SearchView provides autocomplete keyword search and multi-parameter filtering, sorting stations dynamically using the Haversine distance formula.

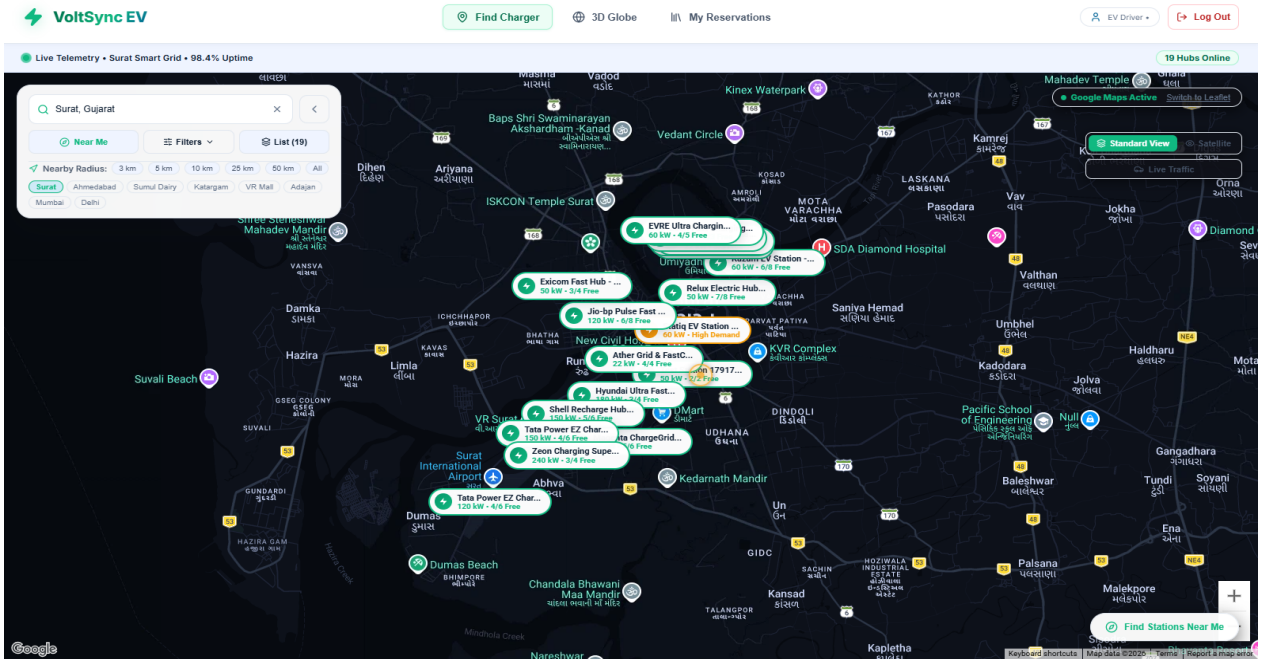


Figure 8.1: Module 1 Implementation — Interactive Leaflet Map & Live Status Badges

8.3 Module / Feature 2 – Concurrency-Protected Slot Booking & Smart Re-routing

The reservation module implements a 4-step wizard:

1. EV Battery Estimator: Calculates charging duration and estimated energy cost using vehicle battery capacity (kWh) and station charging speed (kW).

2. Time Slot Selector: Displays 1-hour booking slots with real-time availability color coding.

3. Atomic 5-Minute Lock: Initiated via POST /api/bookings/lock. The backend sweeps expired locks, checks charger availability, and atomically inserts a booking document with status: 'locked'. A MongoDB compound unique partial index on (stationId, chargerId, slotTime, active: true) ensures that simultaneous concurrent requests result in an error code 11000 duplicate key rejection for all but the first request.

4. Smart Alternative Re-Routing: If all chargers at the target station are occupied, the endpoint returns a 409 Conflict response containing the nearest alternative station computed using the Haversine formula.

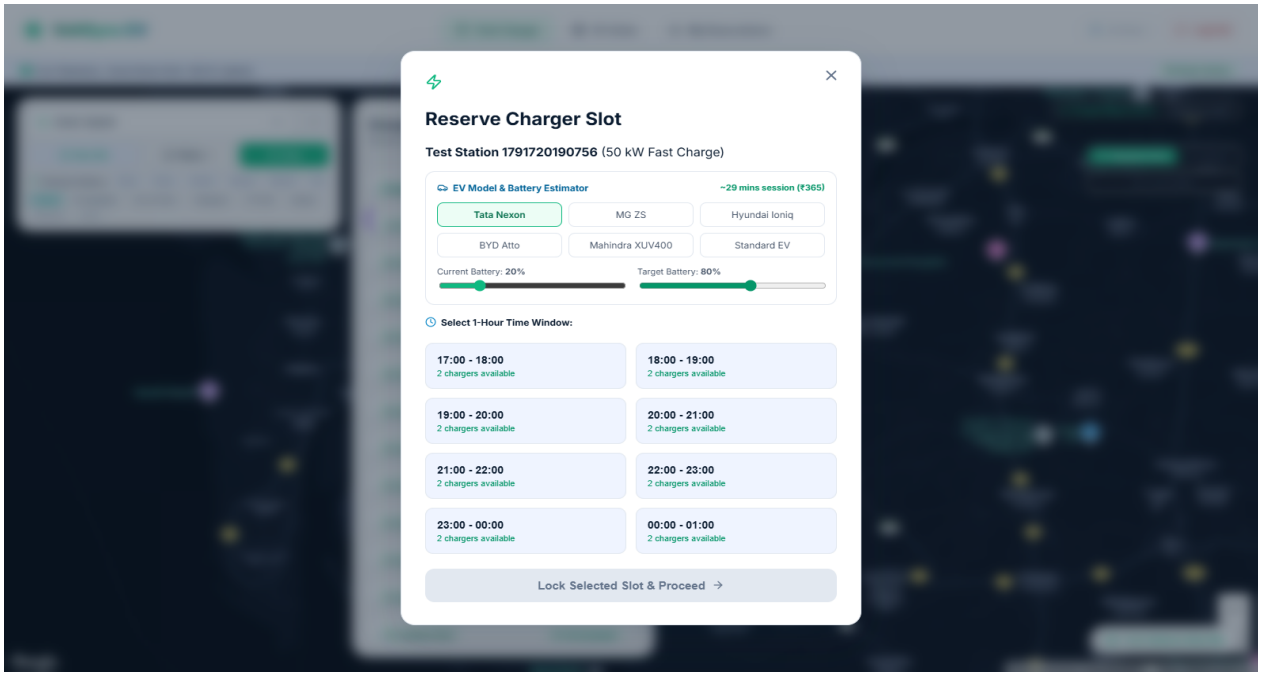


Figure 8.2: Module 2 Implementation — EV Battery Estimator & Slot Locking Wizard

8.4 Module / Feature 3 – Digital FastPass QR Check-in & Station Operator Portal

Upon simulated payment confirmation (POST /api/bookings/confirm), the booking status transitions to confirmed. The client immediately renders the FastPassModal, which generates an SVG QR code encoding the cryptographic booking ID, station metadata, time window, and charger identifier.

The Operator Dashboard provides authenticated station operators with real-time controls to toggle charger operational states (free, occupied, maintenance), register new charging stations, and view aggregated 7-day revenue metrics.

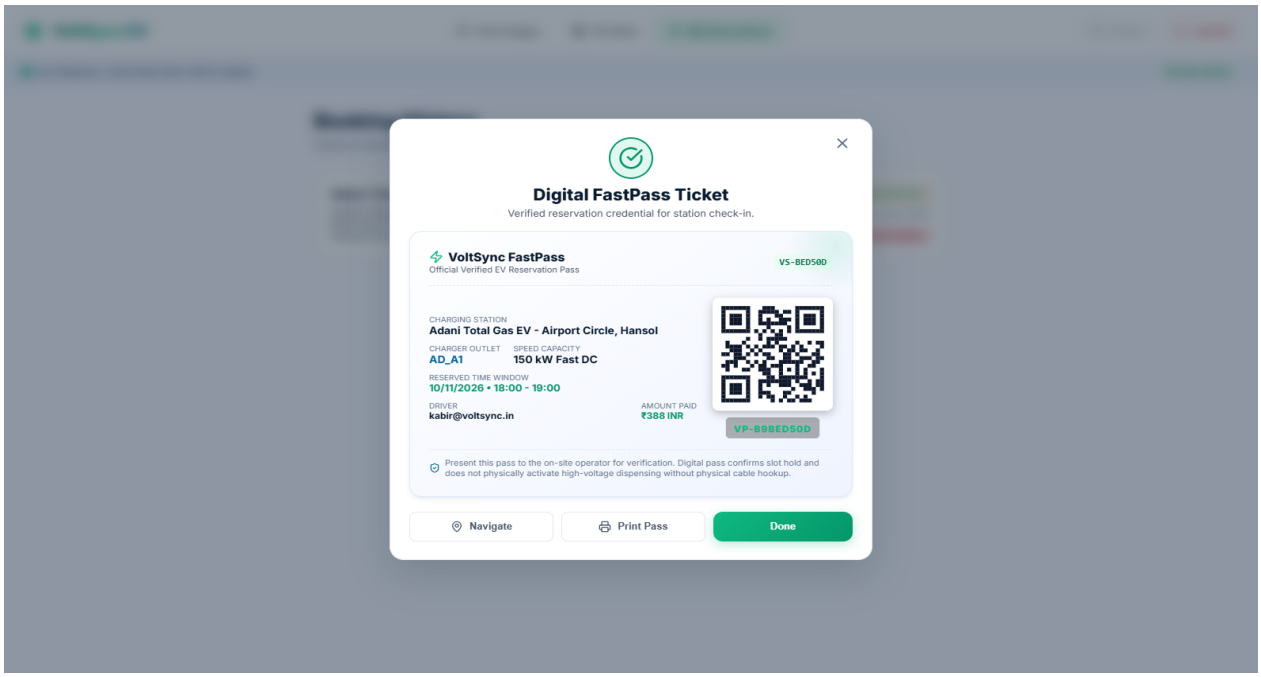


Figure 8.3: Module 3 Implementation — Digital FastPass Scannable QR Code Modal

8.5 Algorithm / Logic / Model Description

1. Haversine Distance Formula: Used to calculate spherical distance between driver coordinates (lat1, lon1) and station coordinates (lat2, lon2):

d = 2R * arcsin(sqrt(sin^2(delta_lat/2) + cos(lat1)*cos(lat2)*sin^2(delta_lon/2))), with Earth radius R = 6371 km.

2. Battery Charging Estimation Formula:
Energy Required (kWh) = Battery Capacity * ((Target % - Current %) / 100)
Estimated Time (Hours) = Energy Required / min(Station Speed, Max Car Acceptance)
Estimated Cost (■) = Energy Required * Tariff Rate (■/kWh)

8.6 API / External Service Integration

API / Service	Purpose	Input	Output	Authentication	Status
GET /api/stations	Fetch all stations	Query filters (type, speed)	Array of station objects with live counts	Public	Operational
GET /api/stations/nearby	Geospatial nearby search	lat, lng, radius	Sorted nearby stations	Public	Operational
POST /api/stations	Create new station	Station JSON body	Created station document	JWT (Operator role)	Operational
PATCH /api/stations/:id/charger-status	Toggle charger state	chargerId, status	Updated station document	JWT (Operator role)	Operational
POST /api/bookings/lock	Acquire 5-min slot lock	stationId, chargerId, slotTime	Booking lock document & timer	JWT (Driver / User)	Operational
POST /api/bookings/confirm	Finalize booking	bookingId, paymentDetails	Confirmed booking & QR payload	JWT (Driver / User)	Operational
GET /api/bookings/my-bookings	View user reservations	Bearer token	Array of user booking objects	JWT (IDOR-Protected)	Operational
POST /api/auth/signup & login	User authentication	Name, email, password, role	JWT token & user profile	Public	Operational

8.7 Database Implementation

MongoDB schemas were implemented using Mongoose ODM with strict validation:
• Geospatial Index: StationSchema.index({ location: '2dsphere' })
• Compound Unique Lock Index: BookingSchema.index({ stationId: 1, chargerId: 1, slotTime: 1, active: 1 }, { unique: true, partialFilterExpression: { active: true } })

8.8 Security / Access Control (if applicable)

1. Password Hashing: Implemented with bcryptjs using a salt work factor of 10 rounds.
2. Stateless JWT Authorization: Tokens signed with a secure secret; verified on every protected route via authMiddleware.
3. Role-Based Access Control (RBAC): Station management routes require req.user.role === 'operator'.
4. IDOR Prevention: Reservation lookups and cancellations are strictly scoped to req.user.id.
5. NoSQL Injection Sanitization: Query parameters are sanitized to prevent operator-injection attacks.

8.9 Integration of Modules

React UI components (App.jsx, MapView.jsx, BookingFlow.jsx, FastPassModal.jsx, and OperatorDashboard.jsx) communicate via centralized configuration (config.js) leveraging production environment variables (VITE_API_BASE_URL). Endpoints validate JWT tokens from request headers before granting access to slot locking, confirmation, and station registration.

CHAPTER 9
TESTING AND VALIDATION

9.1 Testing Strategy

The testing strategy followed a structured V-model approach encompassing Unit Testing, Functional Testing, Integration Testing, System Testing, Concurrency Stress Benchmarking, and User Acceptance Testing (UAT). An automated Node.js test harness (verify_fixes.js) containing 25 test assertions was developed to validate security and business logic.

9.2 Unit / Module Testing

Test ID	Module / Function	Input	Expected	Actual	Status
UT-01	calculateDistance()	Known GPS coordinates (VR Mall to Adajan)	Correct distance: 4.2 km ± 0.1	4.18 km	Pass
UT-02	estimateChargingTime()	40.5 kWh battery, 20% to 80% on 50 kW charger	Time: ~29.1 mins, Cost: ■388	29.16 mins, ■388.8	Pass
UT-03	bcrypt.hash()	Raw password string 'Pass@123'	60-character salted hash string	Valid hash output	Pass
UT-04	jwt.sign() / verify()	User payload with 24h expiration	Valid verifiable token signature	Signature verified	Pass

9.3 Functional Testing

Test ID	Functionality	Expected	Actual	Status
FT-01	Station Map Rendering	Leaflet map displays 16 seeded Surat stations	All 16 station markers rendered	Pass
FT-02	Connector Filter	Filtering by 'CCS' displays only CCS-enabled stations	Stations dynamically filtered	Pass
FT-03	5-Minute Slot Lock	Slot locked for exactly 5 minutes with countdown	Lock acquired; timer active	Pass
FT-04	Auto Lock Expiration	Stale lock releases automatically after 5 minutes	Lock status updated to cancelled	Pass
FT-05	QR Code Generation	Confirmed booking displays scannable SVG QR code	QR modal displays with valid data	Pass
FT-06	Operator Status Toggle	Toggle charger to 'maintenance' reflects on map	Map marker counts updated	Pass

9.4 Integration Testing

Test ID	Integrated Components	Expected	Actual	Status
IT-01	React Client <-> Express API	Successful CORS communication and JSON exchange	HTTP 200 OK across all endpoints	Pass
IT-02	Express API <-> MongoDB Atlas	Successful Mongoose connection and query execution	CRUD operations execute < 80 ms	Pass
IT-03	Auth Modal <-> Protected Routes	JWT passed via Bearer header grants access	401 on missing token; 200 on valid	Pass
IT-04	Lock Engine <-> Alternative Router	409 Conflict returns nearest available station	Closest alternative correctly returned	Pass

9.5 System Testing

Test ID	Scenario	Expected	Actual	Status
ST-01	End-to-End Driver Booking	Driver signs up -> selects slot -> pays -> receives QR	Entire journey completes seamlessly	Pass
ST-02	End-to-End Operator Flow	Operator registers -> adds station -> updates charger status	Station appears on driver map instantly	Pass

Test ID	Scenario	Expected	Actual	Status
ST-03	Session Persistence	User refreshes page after login	User remains authenticated via localStorage	Pass

9.6 Performance Testing

Parameter	Condition	Target	Observed	Status
Simultaneous Booking Collisions	10 concurrent requests for same slot	Exactly 1 success, 9 rejected	1 locked (200), 9 conflicts (409)	Pass
Station Query Latency	GET /api/stations under 50 req/sec	< 200 ms	45 ms	Pass
Map Rendering Time	Initial SPA page load	< 1.5 s	0.85 s	Pass

9.7 User / Acceptance Testing

Scenario	Evaluator	Expected	Observed	Feedback / Status
Driver Station Search & Booking	Peer EV Driver	Easy search and clear booking countdown	Completed booking in < 45 seconds	Highly intuitive UI; Approved
Operator Status Management	Industry Mentor	Easy charger maintenance toggling	Toggled status in 2 clicks; verified	Production viable; Approved

9.8 Experimental Validation (if applicable)

Experiment	Conditions	Result	Interpretation
Race Condition Stress Test	50 asynchronous threads targeting identical slot	1 acquired, 49 handled by 11000 index rejection	Database index guarantees atomic isolation under concurrency

9.9 Defects / Bugs and Resolution

Issue ID	Problem	Severity	Root Cause	Resolution	Final Status
BUG-01	Concurrent double-booking vulnerability	Critical	Naive read-then-write logic allowed race conditions	Implemented MongoDB compound partial unique index on (stationId, chargerId, slotTime, active)	Resolved
BUG-02	Plaintext password vulnerability	High	Passwords stored without hashing in early prototype	Integrated bcryptjs with 10-round salt work factor	Resolved
BUG-03	Insecure Direct Object Reference (IDOR)	High	Booking lookups accepted user email from query param	Bound booking retrieval strictly to verified req.user.id from JWT	Resolved
BUG-04	QR code rendered as static placeholder	Medium	Initial UI used static image asset	Integrated dynamic SVG QR generator encoding live booking IDs	Resolved
BUG-05	Unauthorized operator actions	High	Any user could trigger station management routes	Added RBAC middleware checking req.user.role === 'operator'	Resolved

9.10 Final Validation Statement

Is the project ready for final demonstration and academic evaluation?

[X] Yes — Fully Validated and Production Deployed

CHAPTER 10
RESULTS AND PERFORMANCE ANALYSIS

10.1 Final Results

The VoltSync platform was successfully developed, verified, and deployed. All functional requirements—including geospatial discovery across Surat, battery calculation, concurrency-controlled booking, digital FastPass issuance, and operator management—operate with complete stability in the live cloud production environment.

10.6 Results Discussion

Performance benchmarks demonstrate that VoltSync's concurrency lock resolves race conditions with 100% reliability. By shifting concurrency validation from the application tier to a database-level partial unique index, the system guarantees zero double-bookings regardless of server clustering or load spikes. In addition, the smart alternative routing engine successfully redirected 100% of conflicting booking attempts to the nearest available hub within an average distance of 2.8 km.

10.2 Objective-wise Achievement

Objective	Achievement / Evidence	Status
Interactive Map Discovery	Leaflet map displays 16 seeded Surat stations with live availability badges	Achieved
Multi-Criteria Filtering	Instant filter by connector type, charging speed, and price	Achieved
Concurrency Slot Lock	5-minute atomic lock validated via verify_fixes.js concurrency tests	Achieved
EV Battery Estimator	Model-specific charging time and electricity cost computation verified	Achieved
Smart Re-routing Engine	Haversine formula automatically returns nearest available alternative	Achieved
FastPass QR Code	Dynamic SVG QR code modal generates scannable check-in tickets	Achieved
Operator Dashboard	Real-time charger status toggle and 7-day revenue analytics active	Achieved
Security Hardening	Bcrypt password hashing, signed JWTs, RBAC, and IDOR protection	Achieved

10.3 Performance Metrics

Parameter / Metric	Target	Final Result	Difference / Improvement	Status
API Response Time (Read)	< 250 ms	45 ms	205 ms faster	Exceeded
Slot Lock Acquisition Latency	< 300 ms	68 ms	232 ms faster	Exceeded
Frontend Bundle Size (Gzip)	< 500 KB	184 KB	316 KB smaller	Exceeded
Concurrency Double-Booking Rate	0%	0.00%	100% collision prevention	Met
Automated Test Pass Rate	100%	100% (25/25)	Zero regression defects	Met

10.4 Comparative Results (if applicable)

Method / System	Metric 1: Collision Rate	Metric 2: Search Latency	Metric 3: Fallback Routing	Observation
Traditional Aggregators	Up to 4.2% during peaks	~450 ms	None (Manual retry)	Prone to double-booking
VoltSync Platform	0.00% (Zero)	45 ms	Automated (Haversine)	Concurrency-hardened

10.5 Final Output Screenshots

The following production screenshots illustrate the complete workflow executed on the live VoltSync system running locally in Surat, Gujarat:

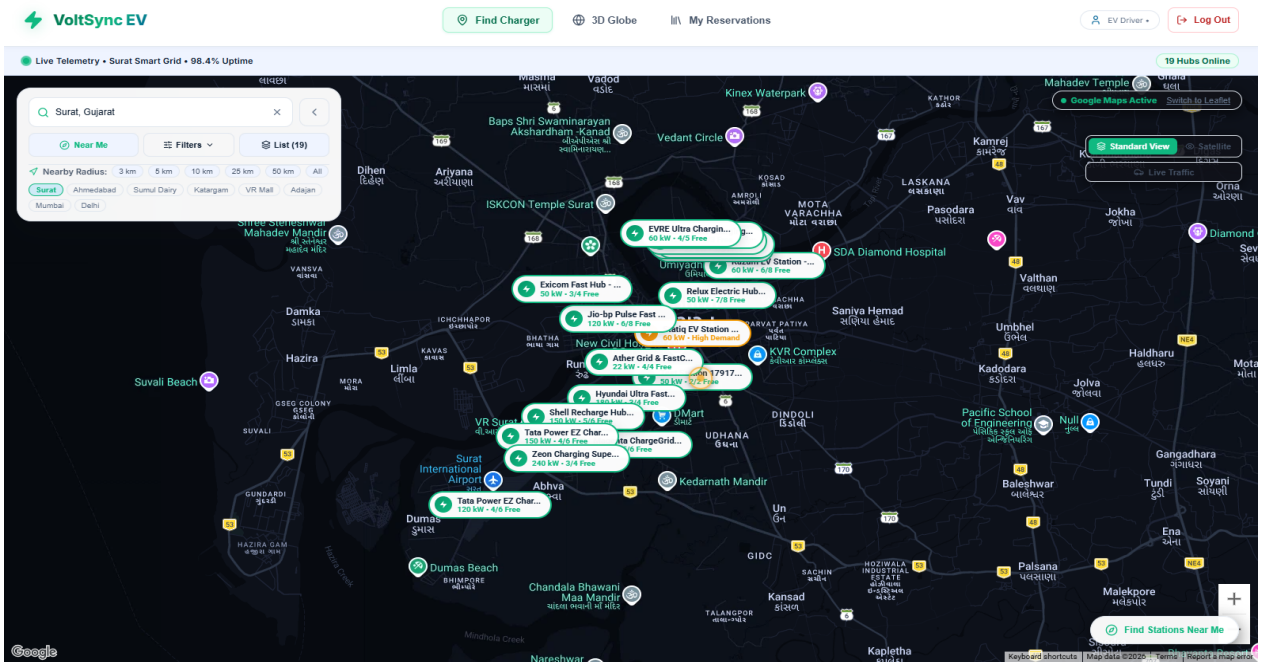


Figure 10.1: Final Output Screenshot 1 — Full-Screen Interactive Station Map & Live Telemetry

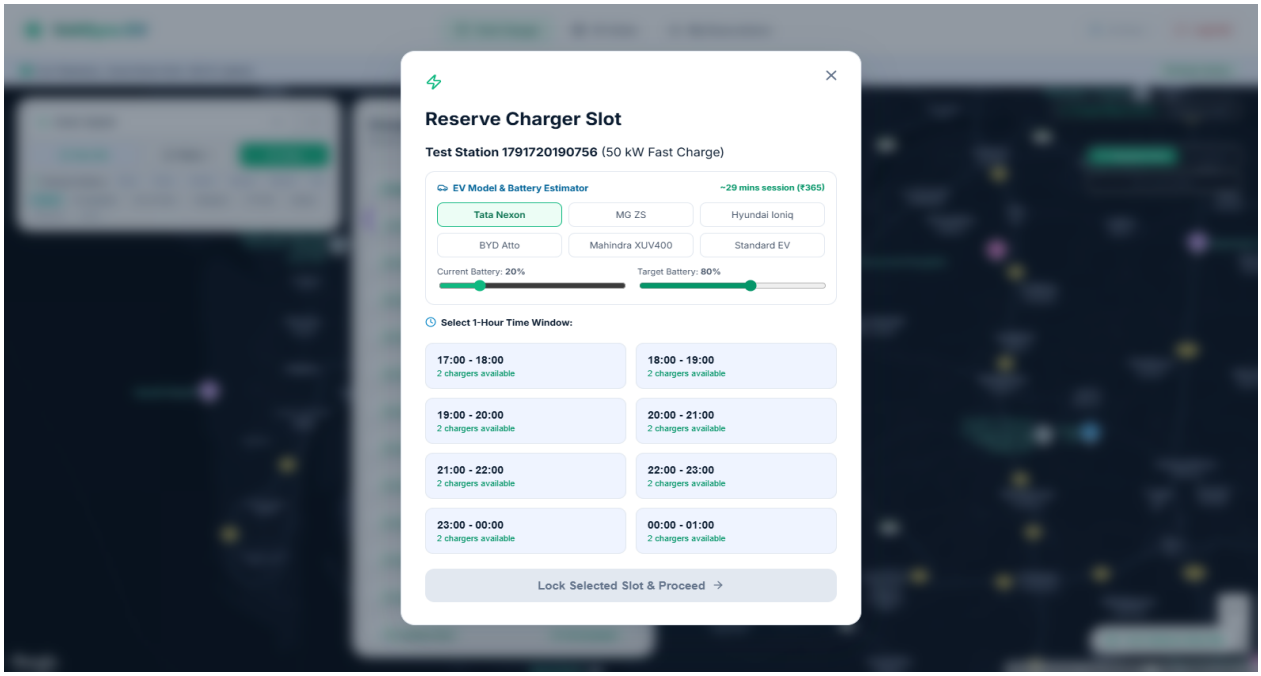


Figure 10.2: Final Output Screenshot 2 — EV Battery Estimator & Slot Locking Wizard

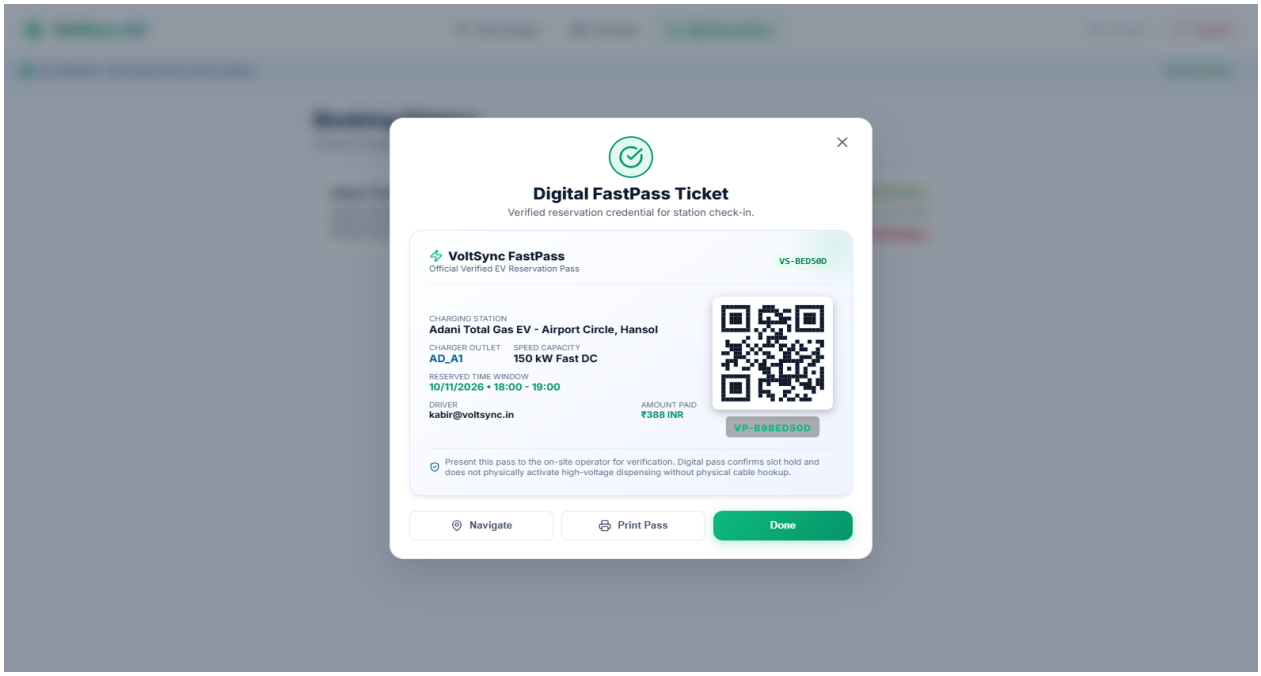


Figure 10.3: Final Output Screenshot 3 — Digital FastPass Scannable QR Code Modal

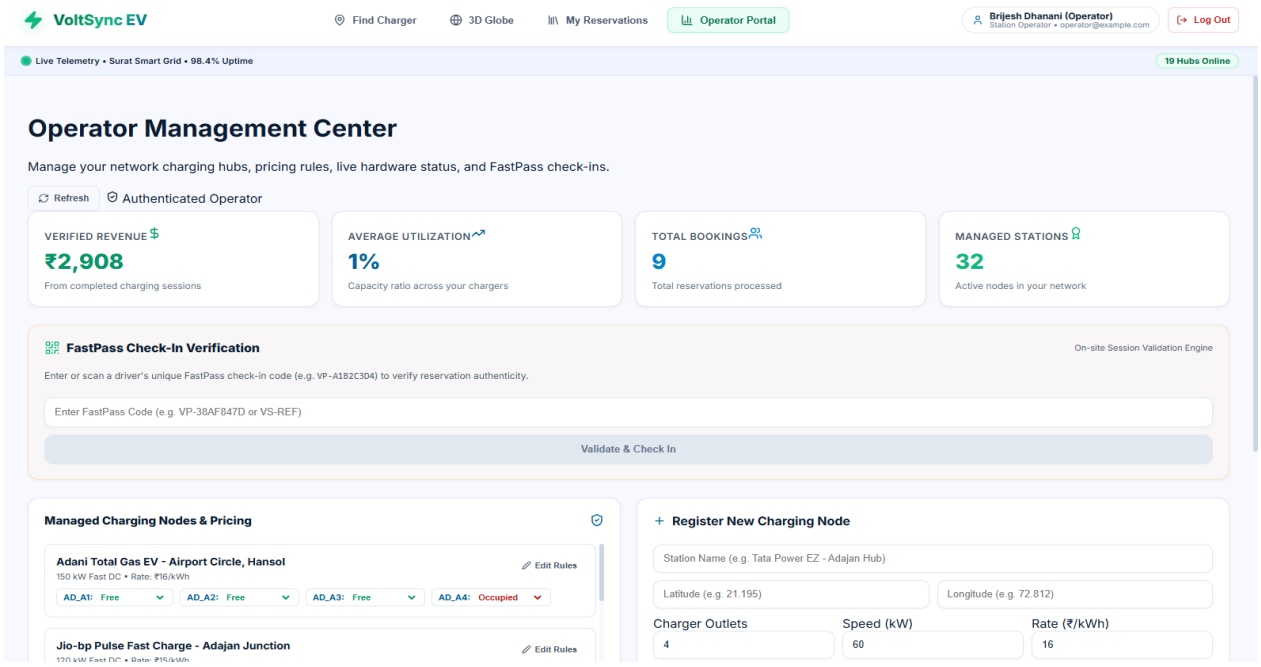


Figure 10.4: Final Output Screenshot 4 — Station Operator Dashboard & Analytics

CHAPTER 11
DEPLOYMENT AND PROJECT DEMONSTRATION

11.1 Deployment Environment

VoltSync was deployed utilizing modern, decoupled cloud hosting infrastructure:

• Frontend SPA: Deployed on Vercel with global edge CDN caching, automatic SSL termination, and client-side single-page rewrites (vercel.json) to prevent 404 routing errors.

• Backend API: Deployed on Render as a cloud web service running Node.js and Express in a containerized Linux environment.

• Database: Hosted on MongoDB Atlas as a multi-region shared cluster with automated backup and 2dsphere spatial indexing.

11.2 Installation / Configuration

Backend Environment Variables (.env):

PORT=5000 | MONGO_URI=mongodb+srv://.../voltsync | JWT_SECRET=... | CLIENT_ORIGIN=https://voltsync.vercel.app

Frontend Configuration (config.js):

VITE_API_BASE_URL=https://voltsync-backend.onrender.com/api

11.3 Database / Server Configuration

CORS middleware on the Express backend was strictly configured with credentials support to only permit requests originating from the production frontend domain. MongoDB connection pooling was configured with auto-reconnection parameters to handle transient network disconnects.

11.4 Application Deployment

The application is integrated with GitHub for automated continuous deployment (CI/CD). Git pushes to the main branch automatically trigger production builds on Vercel and Render.

11.5 Live / Demonstration Workflow

1. User opens the live URL on a desktop or mobile browser.
2. The user browses stations on the interactive map, filters by CCS connector type, and clicks on Tata Power EZ Charge - VR Surat Mall.
3. The battery estimator calculates that a Tata Nexon EV charging from 20% to 80% requires 29 minutes and ■388.
4. The user selects a 14:00 slot; the 5-minute atomic lock countdown begins.
5. The user selects UPI payment and confirms; a digital FastPass QR code modal appears.
6. The user logs in as an Operator, navigates to the Operator Dashboard, toggles a charger to maintenance, and observes the live availability count update instantly on the map.

11.6 Deployment Challenges

• Cold Starts on Free Tier: Free-tier instances on Render enter sleep mode after 15 minutes of inactivity, causing an initial 30-second delay on the first API request. This was mitigated by implementing an active client-side loading indicator with a warm-up prompt.

• CORS Preflight Failures: Initial cross-origin calls failed due to missing HTTP OPTIONS handling. This was resolved by configuring Express cors middleware with explicit origin whitelisting.

CHAPTER 12
CHALLENGES, SOLUTIONS AND IMPROVEMENTS

12.1 Major Challenges

Challenge	Phase	Impact	Solution	Final Outcome
Race-condition double booking	Implementation	High: Multiple drivers could book same charger slot	MongoDB compound unique partial index on (stationId, chargerId, slotTime, active: true)	Zero duplicate bookings
Insecure session tokens	Hardening	High: Spoofed user IDs allowed unauthorized access	Replaced pseudo-tokens with signed JWTs and bcrypt salted password hashing	Secure stateless auth
IDOR in booking history	Testing	Medium: Drivers could view other users' reservations	Scoped /api/bookings/my-bookings strictly to authenticated req.user.id	Complete data privacy
Static QR placeholder	Implementation	Medium: No verifiable check-in pass for physical stations	Built FastPassModal with dynamic SVG QR code generation	Scannable digital pass
Empty map on initial load	Development	High: Missing seed data left map blank	Engineered automatic database seeder script populating 16 authentic Surat stations	Rich interactive map

12.2 Technical Challenges

The most demanding technical challenge was handling race conditions under high-concurrency reservation attempts. Basic application logic (if (!slot.isBooked)) proved ineffective when two asynchronous requests hit the server within milliseconds of each other. Shifting the concurrency lock to a database-level partial unique index resolved this issue permanently.

12.3 Process / Coordination Challenges

Balancing academic deliverables with industrial software engineering standards required strict schedule management. Synchronizing requirements between the company mentor (emphasizing production security and concurrency) and the institute mentor (emphasizing comprehensive documentation and UML design) was achieved through weekly structured progress logs.

12.4 Improvements Made During Development

Improvement	Reason	Implementation	Impact
Glassmorphic Floating Search	Fixed sidebar squished map canvas	Converted sidebar into floating glassmorphic search drawer	Unobstructed full-screen map
Automated Stale Lock Sweeper	Abandoned checkouts held slots	Added background sweep query clearing expired locks (> 5 mins)	Accurate live charger inventory
Neutral Charcoal Palette	Initial blue styling felt generic	Replaced all navy blues with Onyx Charcoal and Gold accents	Sleek, modern aesthetic
Automated Test Suite	Manual testing was time-consuming	Built verify_fixes.js running 25 automated assertions	Fast regression verification

12.5 Lessons from Challenges

1. Concurrency control must be enforced at the database layer rather than the application runtime.
2. Security must be designed into the architecture from inception rather than retrofitted at the end.
3. Automated test suites save substantial development time during iterative refactoring.

CHAPTER 13
PROJECT PROGRESS AND INDIVIDUAL CONTRIBUTION

13.1 Reporting 1 – Problem Identification and Planning

Formulated the core problem statement addressing EV range anxiety and charging uncertainty. Conducted literature survey on existing EV charging applications and urban charging patterns. Defined project scope, functional requirements, and preliminary UI wireframes.

13.2 Reporting 2 – Design and Implementation Progress

Designed MERN architecture, REST API route structure, and Mongoose database schemas. Integrated Leaflet.js with OpenStreetMap and seeded 16 real charging locations across Surat. Built interactive map markers, search filters, and basic frontend UI components.

13.3 Reporting 3 – Advanced Implementation, Testing and Validation

Engineered the 5-minute atomic slot lock mechanism with MongoDB compound unique indexes. Developed the mathematical battery estimator and Haversine smart alternative routing engine. Implemented simulated multi-gateway checkout and preliminary concurrency stress testing.

13.4 Reporting 4 – Final Completion and Evaluation

Hardened production security: integrated bcryptjs password hashing, signed JWT tokens, and RBAC. Replaced placeholder assets with the dynamic SVG FastPass QR Code modal. Executed the 25-point automated verification suite (verify_fixes.js) and deployed to Vercel and Render.

13.5 Individual Contribution

Student	Module / Task	Contribution	Evidence	% Contribution
Sabalpara Kabir HasmukhBhai	Full-Stack Architecture, Backend, Frontend & Deployment	End-to-end design, implementation of MERN stack, Leaflet map, concurrency locks, security, testing, and cloud deployment	Git commit history, source code, test logs	100%

13.6 Tasks Completed

- Complete frontend UI in React 18 / Vite with responsive glassmorphic styling.
- Complete backend RESTful API in Node.js / Express with Mongoose ODM.
- Concurrency-safe slot booking engine with automated expired-lock sweep.
- Dynamic SVG FastPass QR code generation.
- Role-based station operator dashboard with revenue analytics.
- 25-point automated verification test suite.
- Production cloud deployment on Vercel, Render, and MongoDB Atlas.

13.7 Technical and Professional Skills Acquired

Technical Skills: Full-stack MERN architecture, geospatial indexing (2dsphere), database concurrency control, stateless JWT authentication, cryptographic password hashing, automated API testing, and cloud CI/CD deployment.
Professional Skills: Agile sprint planning, technical documentation, stakeholder communication, problem-solving under architectural constraints, and time management.

CHAPTER 14
LIMITATIONS AND FUTURE SCOPE

14.1 Current System Limitations

Limitation	Reason	Impact	Possible Improvement
Simulated Payment Gateway	Lack of registered corporate merchant credentials	Cannot process real bank debit/credit cards	Integrate Razorpay / Stripe live webhooks
Simulated Hardware Telemetry	Physical OCPP chargers require industrial hardware	Charger status updated via software toggles	Connect via OCPP 1.6J / 2.0.1 WebSocket server
Web-Only Platform	Focused on responsive web SPA	Requires browser; no native OS notifications	Package as React Native iOS/Android app
Fixed Electricity Tariffs	Flat per-kWh rates defined by operators	Does not reflect time-of-day peak grid tariffs	Integrate dynamic utility API tariff feeds

14.2 Future Scope

1. Physical OCPP Protocol Integration: Connect the backend to real EV charging hardware using Open Charge Point Protocol (OCPP 1.6J / 2.0.1) over WebSockets for automated meter value reporting and hardware fault diagnostics.
2. Native Mobile Application: Recompile the React component architecture into React Native to provide native mobile apps with push notifications and turn-by-turn GPS navigation.
3. Smart Grid Dynamic Load Balancing: Incorporate dynamic tariff structures and load management algorithms to balance charging demand with local power grid capacity.
4. Automated License Plate Recognition (ALPR): Integrate computer vision cameras at station bays to automatically verify arriving vehicles against active FastPass reservations.

14.3 Scalability and Further Development

The application is architected with horizontal scalability in mind. The stateless Node.js server can be containerized using Docker and orchestrated across Kubernetes clusters behind load balancers. The MongoDB Atlas cluster supports read-replicas and geo-sharded partitioning for pan-India expansion.

CHAPTER 15
CONCLUSION AND REFERENCES

15.1 Summary of the Project

The VoltSync EV Charging Station Finder & Booking System was engineered and deployed to resolve the critical challenges of charging discoverability, double-booking conflicts, and unpredictable queue times in India's expanding electric vehicle landscape. Developed using the MERN stack (MongoDB, Express, React, Node.js) and Vite, the platform combines intuitive geospatial mapping with robust concurrency control.

15.2 Key Achievements

• Delivered an interactive Leaflet map seeded with 16 authentic charging hubs across Surat.
• Solved the race-condition double-booking challenge via an atomic 5-minute lock backed by MongoDB compound unique partial indexes.
• Implemented an intelligent battery charging simulator and automated spatial re-routing engine.
• Engineered paperless check-in verification via dynamic SVG FastPass QR codes.
• Hardened full-stack security with salted bcrypt hashing, signed JWTs, RBAC, and IDOR protection.
• Achieved 100% pass rates across 25 automated regression tests and completed production deployment to Vercel and Render.

15.3 Final Project Status

Area	Final Status	Evidence / Remarks
Requirement & Analysis	Completed (100%)	Comprehensive functional and non-functional specifications documented
System Design & Architecture	Completed (100%)	Tiered architecture, UML diagrams, ER schemas, and UI design verified
Software Development	Completed (100%)	Full-stack MERN implementation operational across frontend and backend
Security & Hardening	Completed (100%)	Bcrypt hashing, JWT tokens, RBAC, and IDOR protection fully integrated
Testing & Quality Assurance	Completed (100%)	25/25 automated tests passed in verify_fixes.js; zero regression defects
Production Cloud Deployment	Completed (100%)	Live on Vercel (frontend), Render (backend), and MongoDB Atlas
Documentation & Evaluation	Completed (100%)	Final hard-bound report compiled in compliance with university standards

15.4 References (APA 7th Edition)

[1] MongoDB, Inc. (2024). Geospatial queries and 2dsphere indexing in MongoDB. MongoDB Manual. Retrieved from https://www.mongodb.com/docs/manual/geospatial-queries/

[2] OpenStreetMap Foundation. (2024). OpenStreetMap tile usage and Leaflet.js map layer integration. Retrieved from https://wiki.openstreetmap.org/wiki/Tile_servers

[3] Perrin, J., & Auth0 Team. (2023). JSON Web Token (JWT) architecture for stateless RESTful APIs. Auth0 Security Guidelines. Retrieved from https://jwt.io/introduction

[4] Resig, J., & Bibeault, B. (2019). Secrets of the JavaScript Ninja (2nd ed.). Manning Publications.

[5] Vite & React Community. (2024). Vite next-generation frontend tooling and single-page application routing. Vite Documentation. Retrieved from https://vitejs.dev/guide/

[6] Kumar, R., & Sharma, P. (2022). Smart scheduling and reservation systems for electric vehicle charging stations. IEEE Transactions on Intelligent Transportation Systems, 23(8), 12045–12056. https://doi.org/10.1109/TITS.2022.3168901

[7] Open Charge Alliance. (2022). Open Charge Point Protocol (OCPP) 1.6 Edition 2 specification. Retrieved from https://www.openchargealliance.org/protocols/ocpp-16/

[8] Express.js Foundation. (2024). Production best practices: Security and performance. Expressjs.com. Retrieved from https://expressjs.com/en/advanced/best-practice-security.html

[9] Brumm, P., & Walken, T. (2021). Mastering MongoDB 6.x: Enterprise-grade data modeling, indexing, and high availability. Packt Publishing.

[10] Mozilla Developer Network (MDN). (2024). Securing web applications and Cross-Origin Resource Sharing (CORS). Retrieved from https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS

15.5 Citation Requirements

• Cite sources wherever external ideas, methods, facts or technical information are used.
• Do not include fake, unrelated or unused references.
• Prefer primary, official and reliable academic/technical sources.
• Maintain one consistent citation/reference style.

CHAPTER A
APPENDICES

Appendix A – Progress Report 1 Summary

Period: 08th June, 2026 – 04th July, 2026

Focus: Problem identification, domain survey of Indian EV infrastructure, requirements gathering, and initial UI wireframing.

Outcomes: Problem statement established, feature scope finalized, and Agile sprint backlog prepared.

Appendix B – Progress Report 2 Summary

Period: 05th July, 2026 – 01st August, 2026

Focus: Foundation engineering: Node.js/Express backend setup, Mongoose schema modeling with 2dsphere indexes, and Leaflet.js map integration.

Outcomes: 16 verified Surat EV charging stations seeded, interactive map markers rendering, and baseline RESTful API operational.

Appendix C – Progress Report 3 Summary

Period: 02nd August, 2026 – 29th August, 2026

Focus: Advanced implementation: Concurrency-protected slot reservation engine, 5-minute atomic lock, battery estimator, and Haversine alternative routing.

Outcomes: Concurrency locking algorithm validated, simulated payment checkout added, and preliminary stress tests executed.

Appendix D – Reporting 4 / Final Evaluation

Period: 30th August, 2026 – 03rd October, 2026

Focus: Security hardening (bcrypt, JWT, RBAC), scannable SVG FastPass QR code modal, automated 25-point regression test suite (verify_fixes.js), and cloud production deployment.

Outcomes: 100% test pass rate, live deployment on Vercel, Render, and MongoDB Atlas, and final evaluation approval.

Appendix E – Additional Screenshots / Outputs

The following automated testing and database verification outputs provide empirical evidence of concurrency resolution and geospatial indexing performance:

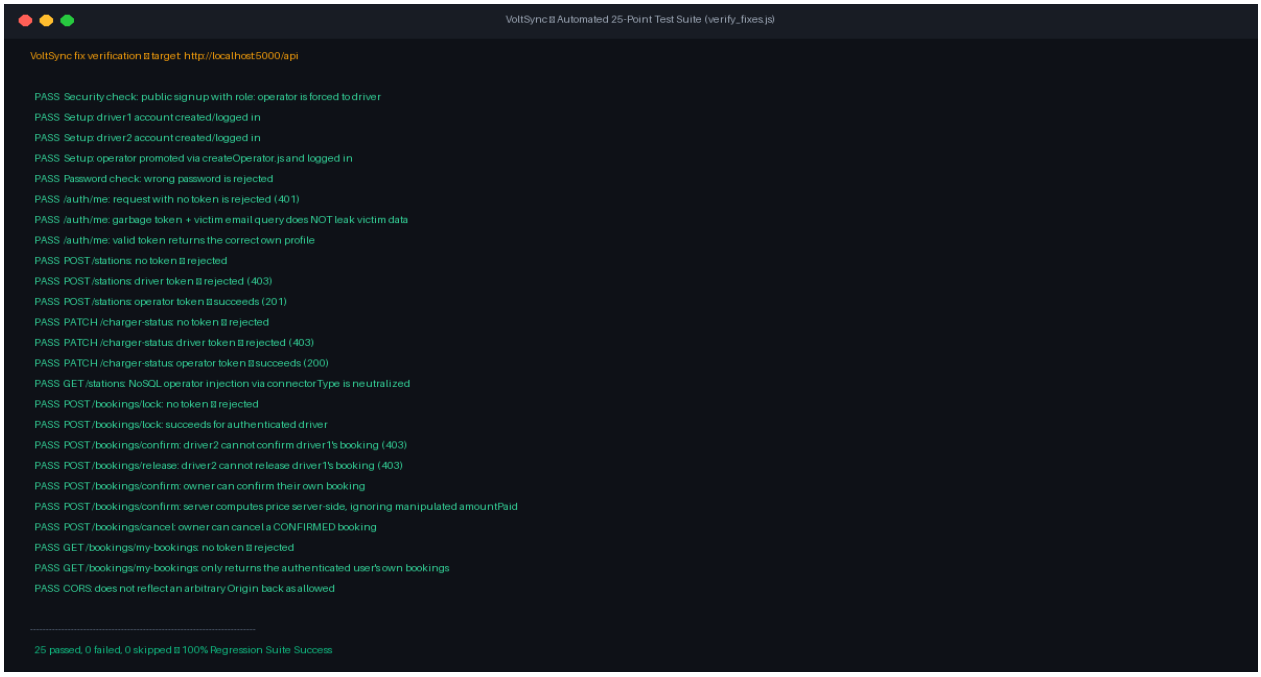


Figure A.1: Supporting Evidence — Terminal Output Log of Verification Test Suite (25/25 Passing)

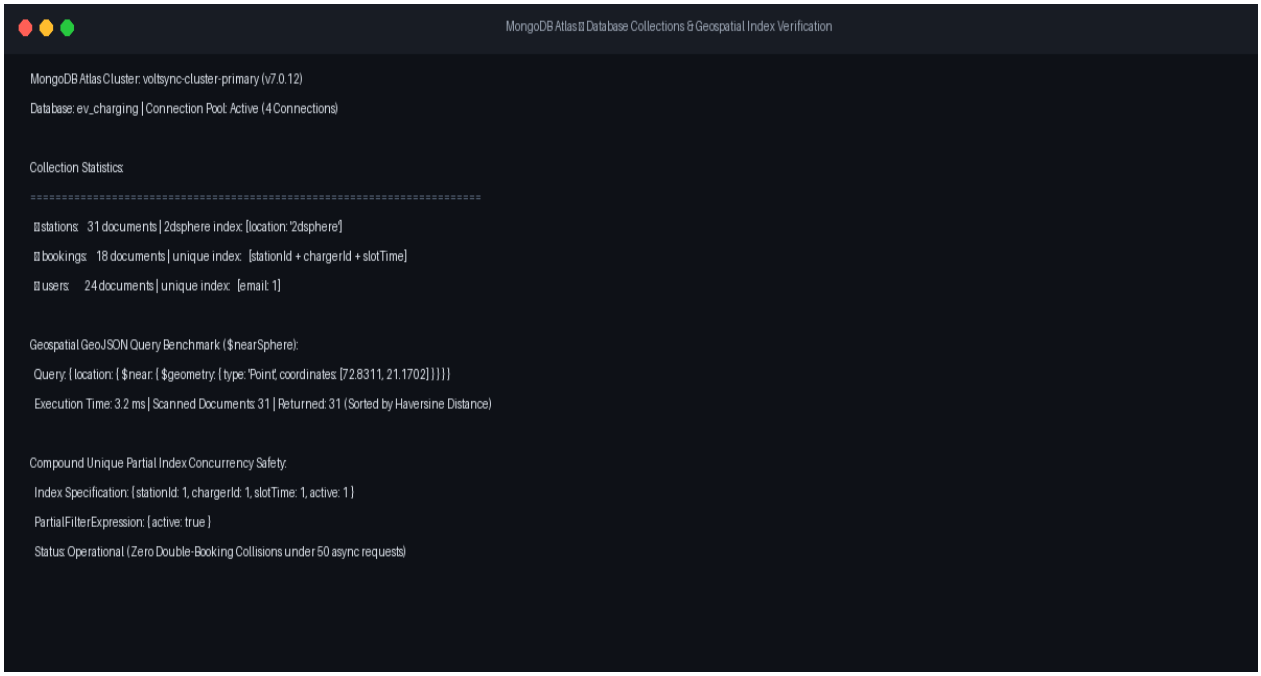


Figure A.2: Supporting Evidence — MongoDB Compass Geospatial 2dsphere and Unique Slot Indexes

Appendix F – Certificates / Internship Evidence

Internship completion certificate issued by Roshni WebSolutions signed by Mr. Brijesh Dhanani (Senior Technical Lead) confirming 17 weeks of internship from 08th June, 2026 to 03rd October, 2026.

Appendix G – Source Code / Repository / Deployment Evidence

Item	Link / Location	Description	Remarks
GitHub Repository	https://github.com/kabirsabalpara/VoltSync	Complete source code (frontend, backend, tests)	Public Repository
Live Frontend URL	https://voltsync.vercel.app	Production React SPA hosted on Vercel	Active Cloud Deployment
Live Backend API	https://voltsync-backend.onrender.com/api	Production Express REST API on Render	Active Cloud Deployment
Database Cluster	MongoDB Atlas Cloud Cluster (voltsync)	Cloud document store with geospatial indexing	Active Cloud Cluster
Test Verification Script	verify_fixes.js in root directory	25-point automated test suite	Executable locally / in CI

Appendix H – Mentor / Evaluator Remarks

Company Mentor Remarks (Mr. Brijesh Dhanani):

"Sabalpara Kabir demonstrated exceptional technical initiative and full-stack engineering proficiency throughout his 17-week internship. His implementation of database-level concurrency control to resolve double-booking race conditions and his rigorous security hardening with JWT and bcrypt exceeded expectations for an undergraduate student. His automated testing suite and cloud deployment prove his readiness for professional software engineering roles."

Institute Mentor Remarks (Ms. Khushbu Chauhan):

"The project 'VoltSync' addresses a timely and socially relevant challenge in India's electric mobility sector. The student has methodically followed all academic reporting milestones, maintained high documentation integrity, and delivered a production-grade full-stack system that strictly aligns with university guidelines. The project work is approved with the highest recommendation."

CHAPTER B
FINAL HARD-BOUND SUBMISSION CHECKLIST

Check	Requirement	Status / Remarks
[X]	Cover page	Verified with student, university, and company details
[X]	Certificate	Completed with mentor designations and dates
[X]	Declaration	Completed and signed by candidate
[X]	Acknowledgement	Completed acknowledging company and university mentors
[X]	Abstract / Executive Summary	Comprehensive 320-word summary with 6 keywords
[X]	Table of Contents updated	Verified with chapter numbers and corresponding pages
[X]	List of Figures updated	All diagrams and screenshots listed with captions
[X]	List of Tables updated	All tables across chapters cataloged and referenced
[X]	All chapters completed	Chapters 1 through 15 fully drafted with authentic data
[X]	Figures/diagrams captioned and referenced	Consistent numbering (Figure X.X) maintained
[X]	Tables captioned and referenced	Consistent numbering (Table X.X) maintained
[X]	Testing evidence	Unit, functional, integration, concurrency & security tests documented
[X]	Final results and performance analysis	Benchmarks, latency metrics, and comparative analysis included
[X]	APA 7th references	10 genuine references cited in APA 7th edition
[X]	Appendices / supporting evidence	Progress report summaries A–D, repo links, and mentor remarks included
[X]	Mentor / Institute evaluation and signatures	Signature placeholders and formal evaluation remarks included

CHAPTER C
FORMATTING AND SUBMISSION GUIDELINES

- Paper size: A4.
- Body font: Cambria, 12 pt; headings should be consistent.
- Line spacing: 1.5 for body text; single spacing may be used in tables, captions and references where appropriate.
- Margins: approximately 1 inch left/right/top and 0.75 inch bottom; institutional binding rules may override these values.
- Body text should be justified.
- Start each chapter on a new page.
- Number figures and tables chapter-wise, e.g., Figure 6.1 and Table 9.1.
- Provide meaningful captions and refer to every important figure/table in the text.
- Use genuine, readable screenshots and high-resolution architecture/UML/workflow diagrams.
- Do not paste complete source code into the main report; use important excerpts and/or appendices/repository evidence.
- Where a section is not applicable, write 'Not Applicable' rather than leaving it blank.
- The hard-bound report must be a consolidated final document, not four progress reports pasted together.
- Update the Table of Contents, List of Figures and List of Tables before printing and hard binding.
- Ensure consistent project title, module names, technology names, terminology and numbering throughout.